

IoT: Architecture and Protocol

Chapter 3: Interoperability

3.1 Applications in IoT

3.2 REST architecture

3.3 Web of things

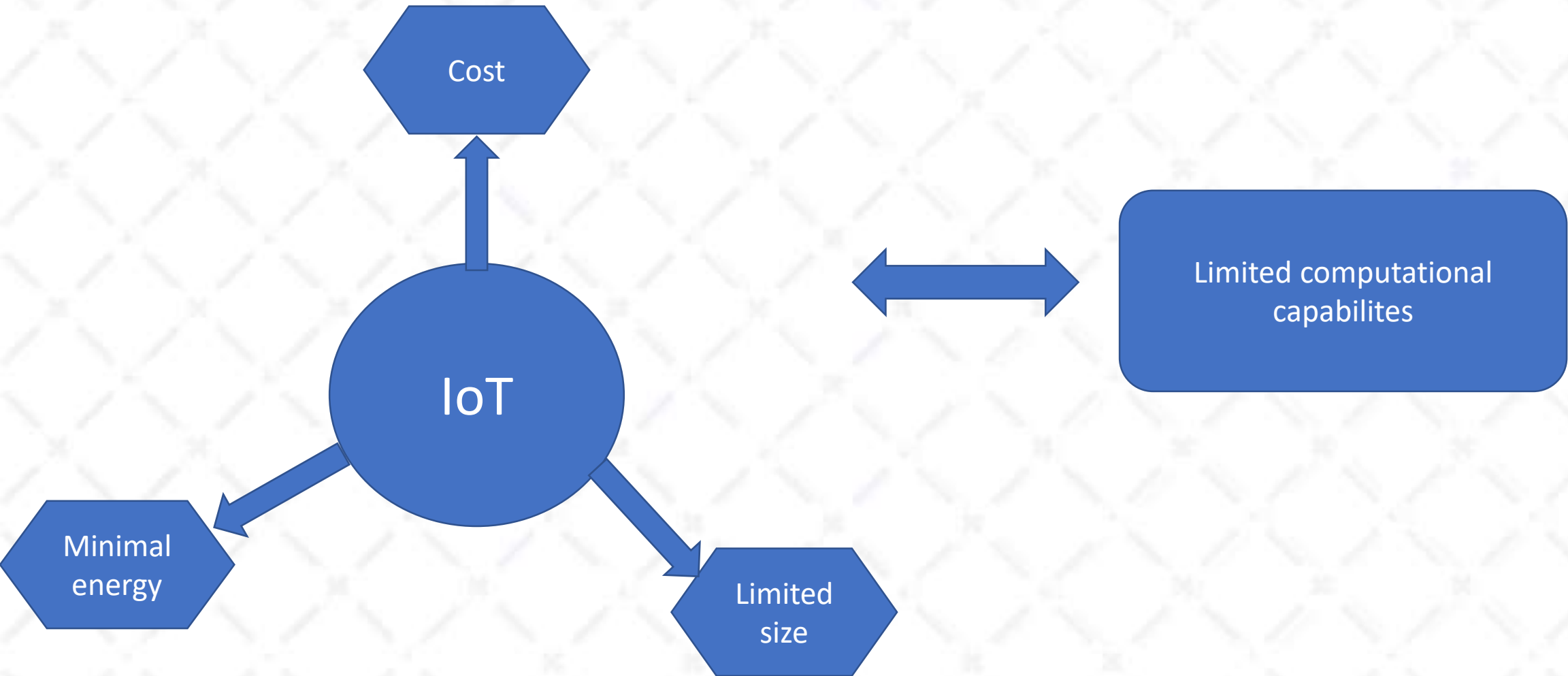
3.4 Messaging Queues and Publish/Subscribe Communications

3.5 Session Initiation for the IoT

3.6 Performance Evaluation

3.7 Optimized Communications: the Dual-network Management Protocol

3.1 Applications in IoT



3.1 Applications in IoT

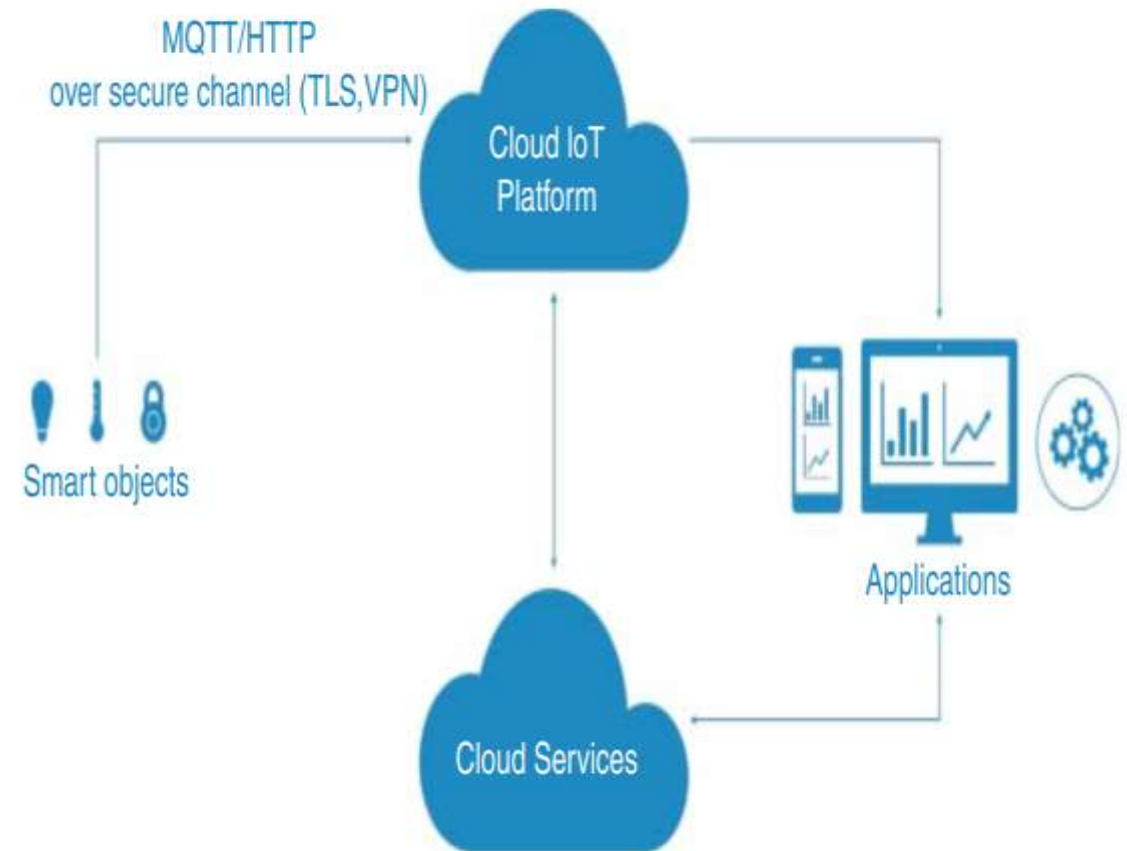
- Limited computational capabilities
 - Be hard to process large messages
 - lower energy consumption

⇒ minimize the amount of transmitted data.
- Operate in low-power and lossy networks (LLNs)
 - Benefit from lightweight protocols
- Communication protocols can be classified into two categories
 - request/response
 - publish/subscribe (pub/sub)

Cloud-based Solutions

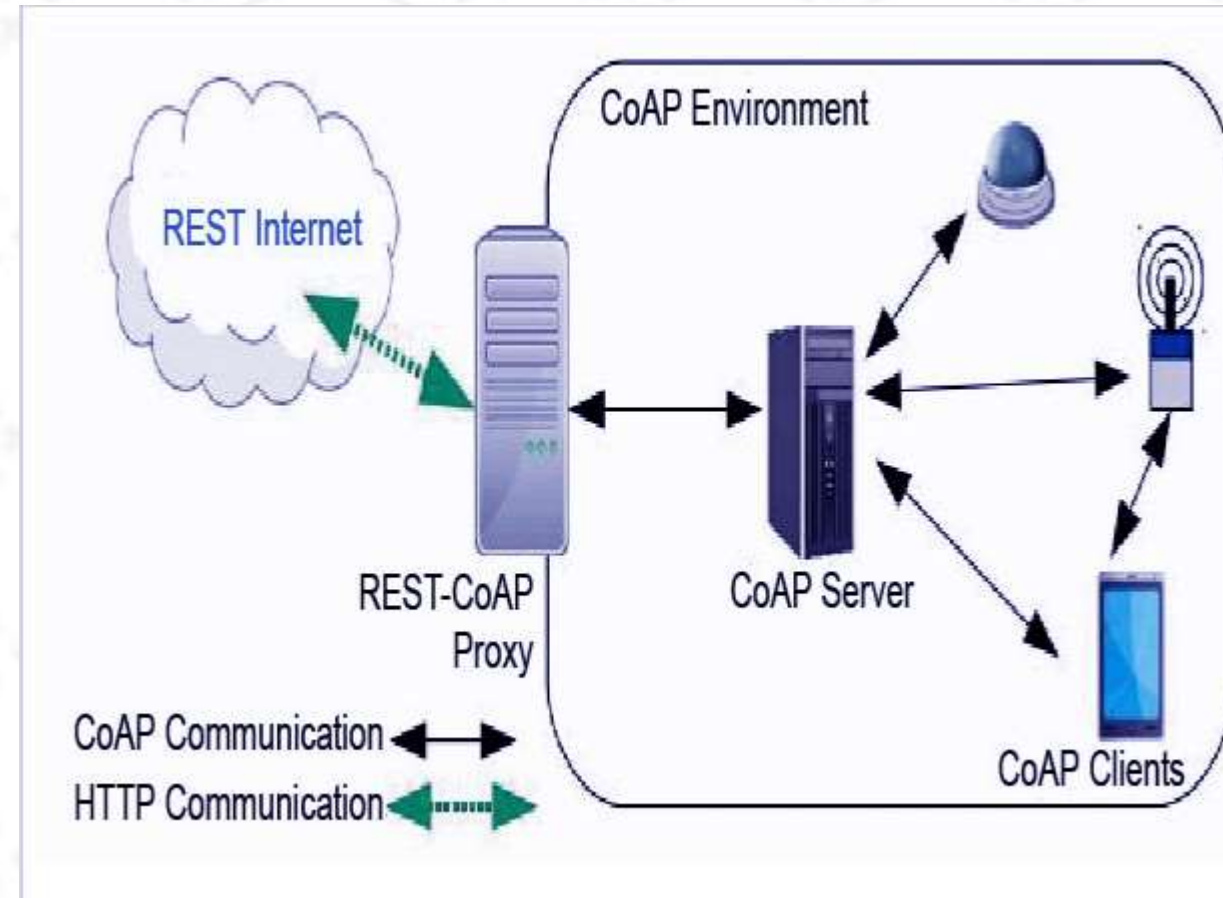
- Cloud service
 - Provide a storage facility for all data
 - a HTTP-based interface for access by clients

⇒ not enough to actually create a worldwide network of interconnected devices
- To build a long-term design:
 - *Scalability*: networks and services ready to handle the traffic generated by billions of things?
 - *Availability*: What happens if an Internet connection becomes temporarily or permanently unavailable?
 - *Interoperability*: do not allow direct interactions among things made by different manufacturers
 - *Security*: attackers could exploit a breach in the cloud
 - *Evolution of systems*: require a software/ firmware upgrade to change how they operate



3.2 REST Architecture: The Web of Things

- Web-based approach in the IoT
 - Be around for decades and IoTs of experience has been gained
 - Store documents, resources, and to build distributed applications
- Uniform resource identifiers (URIs) and the Hypertext Transfer Protocol (HTTP)
 - Be application-layer protocol for hypermedia systems
 - Hypertext Markup Language (HTML)
 - Javascript
 - Cascading Stylesheets (CSS)



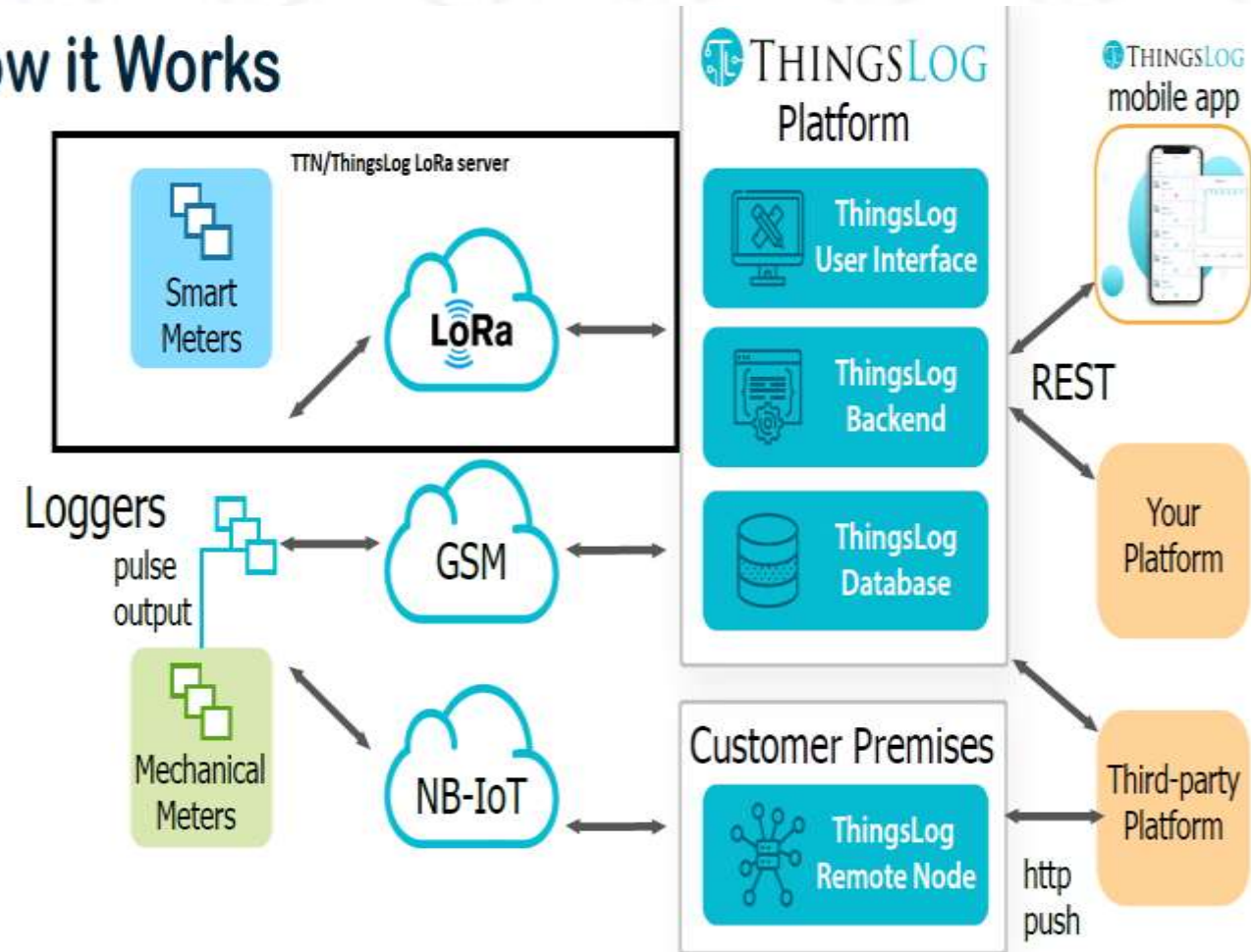
REST, MQTT, CoAP

3.2 REST: The Web as a Platform

- REpresentational State Transfer (REST)
 - Architectural style behind the web
 - Defines a set of rules and principles
 - Scalability (number of interacting clients)
 - Robustness (long-term evolution of systems).
 - Missing information be collected while interacting
 - client knows basic information
 - server drive the client

=> any change on the server must be matched by a change in the client

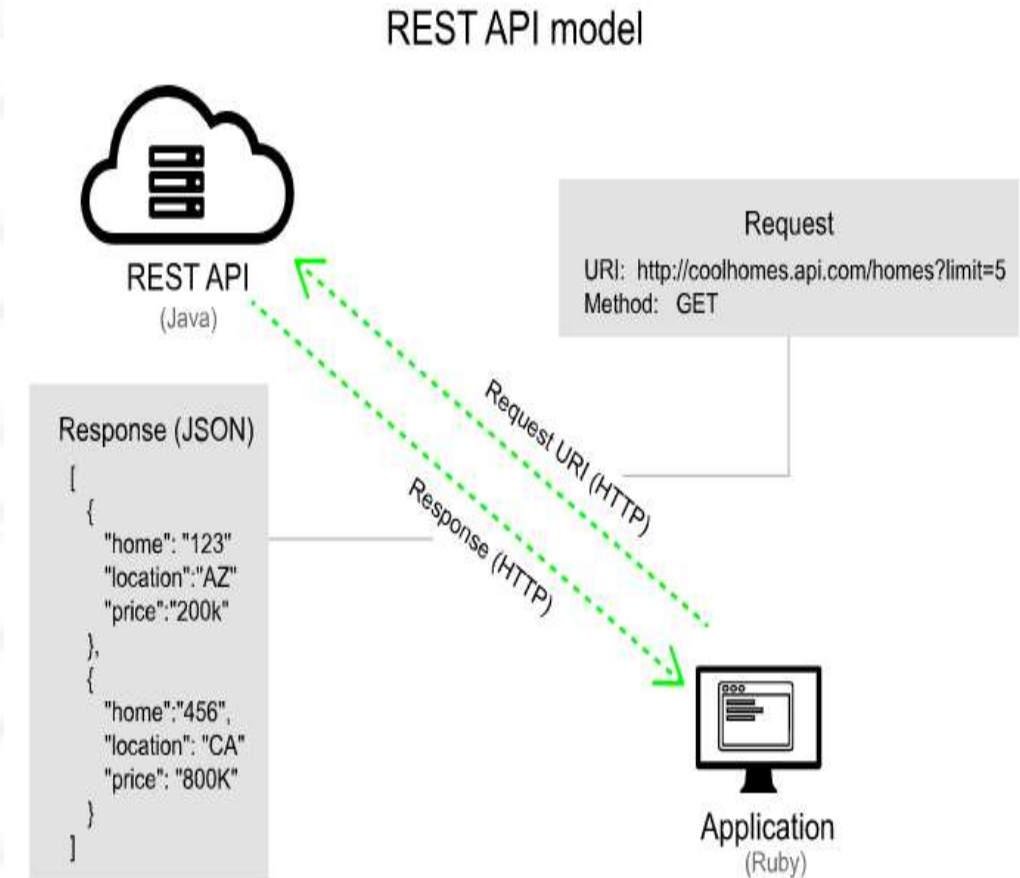
How it Works



3.2 REST: The Web as a Platform

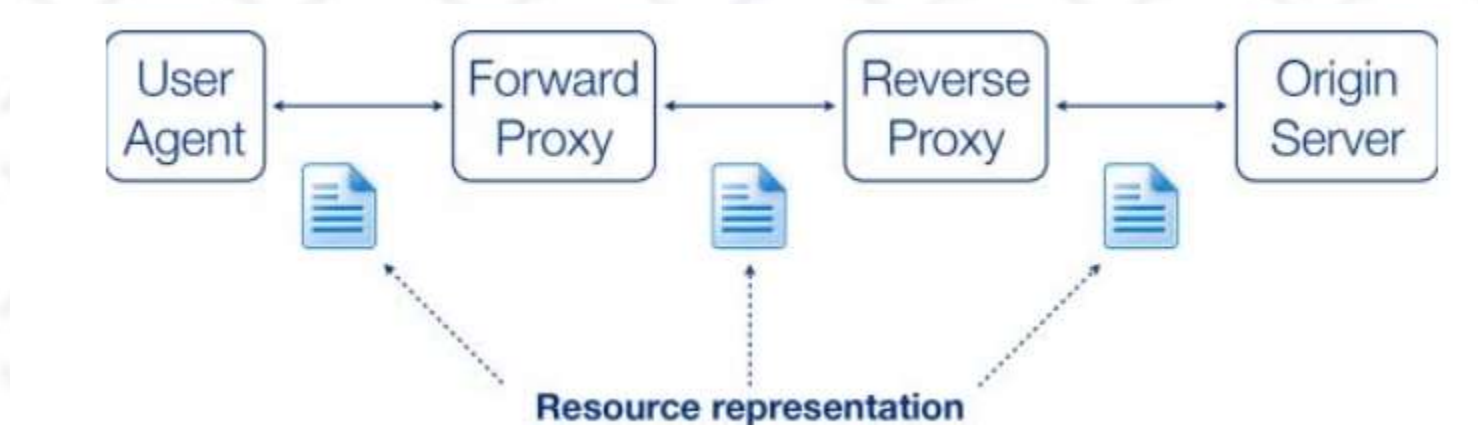
- REST is based on the concept of a web resource
 - webpage, a video, and an order on an e-commerce website
- REST architectures
 - *clients* (or user agents, such as browsers)
 - the application interface and initiate the interactions
 - *servers* (origin servers)
 - host resources and serve client requests
- REST defines the following principles
 - identification of resources
 - manipulation of resources through representations
 - self-descriptive messages
 - hypermedia as the engine of application states

RESTful



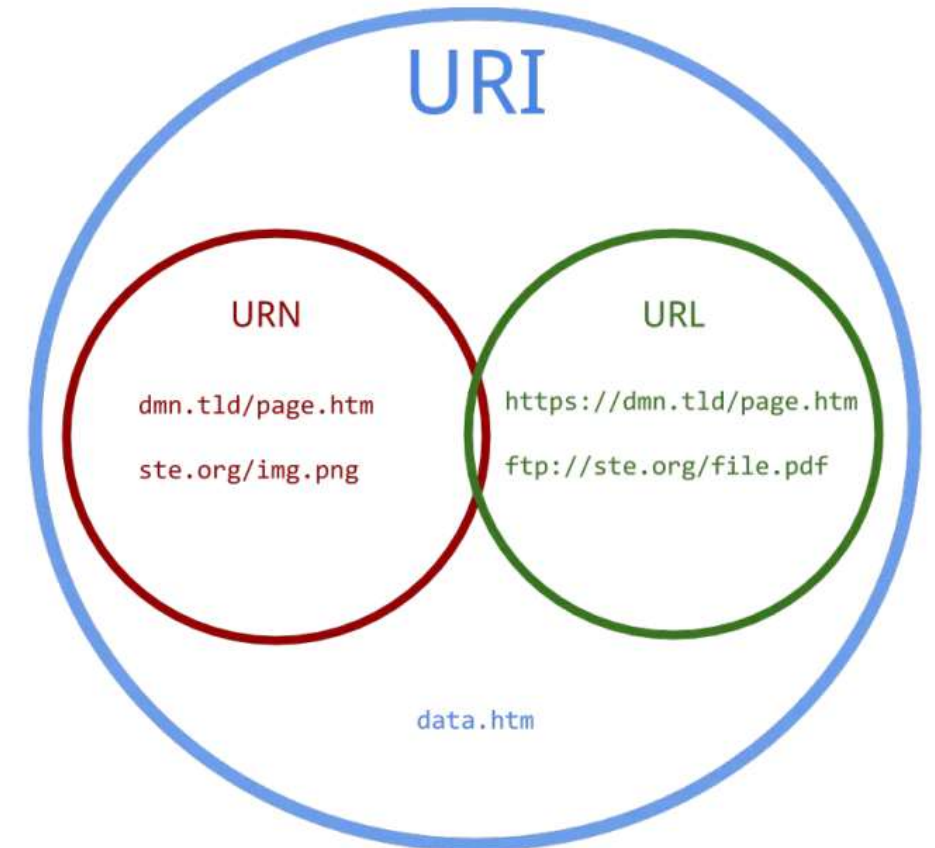
Representation of Resources

- Exchange between endpoints
- Encoded in one or more transferable formats (XHTML, Atom, XML, JSON, plain text, comma-separated values, MP3, or JPEG)



Resource Identifiers

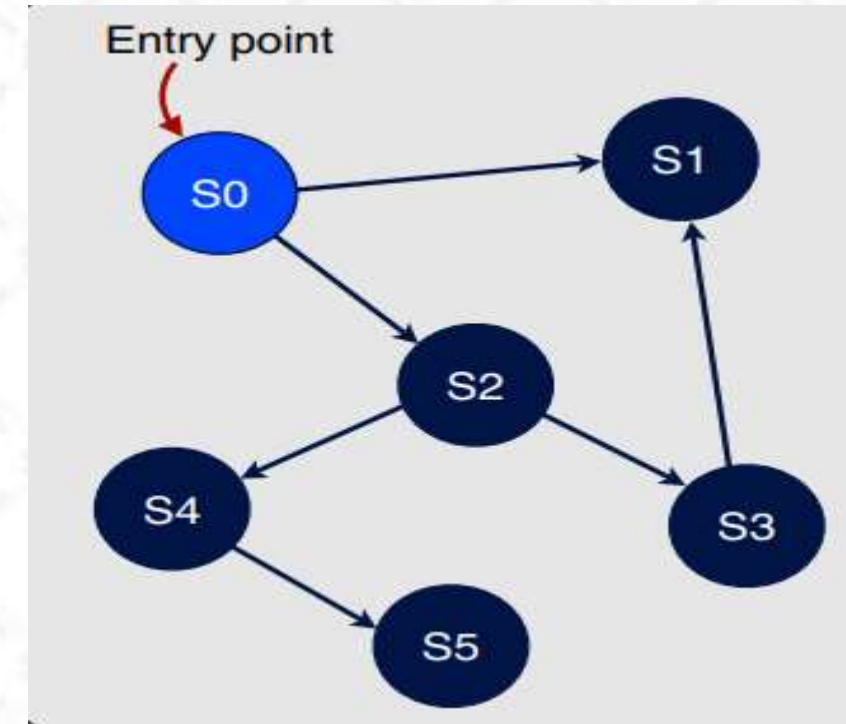
- To ensure correct resource for application
 - Uniform resource identifiers (URIs)
 - be used to address a resource
 - be located, retrieved, and manipulated
 - a uniform resource name (URN) specifies the name of a resource
 - a uniform resource locator (URL) specifies how to locate the resource



Applications as Finite-state Machines

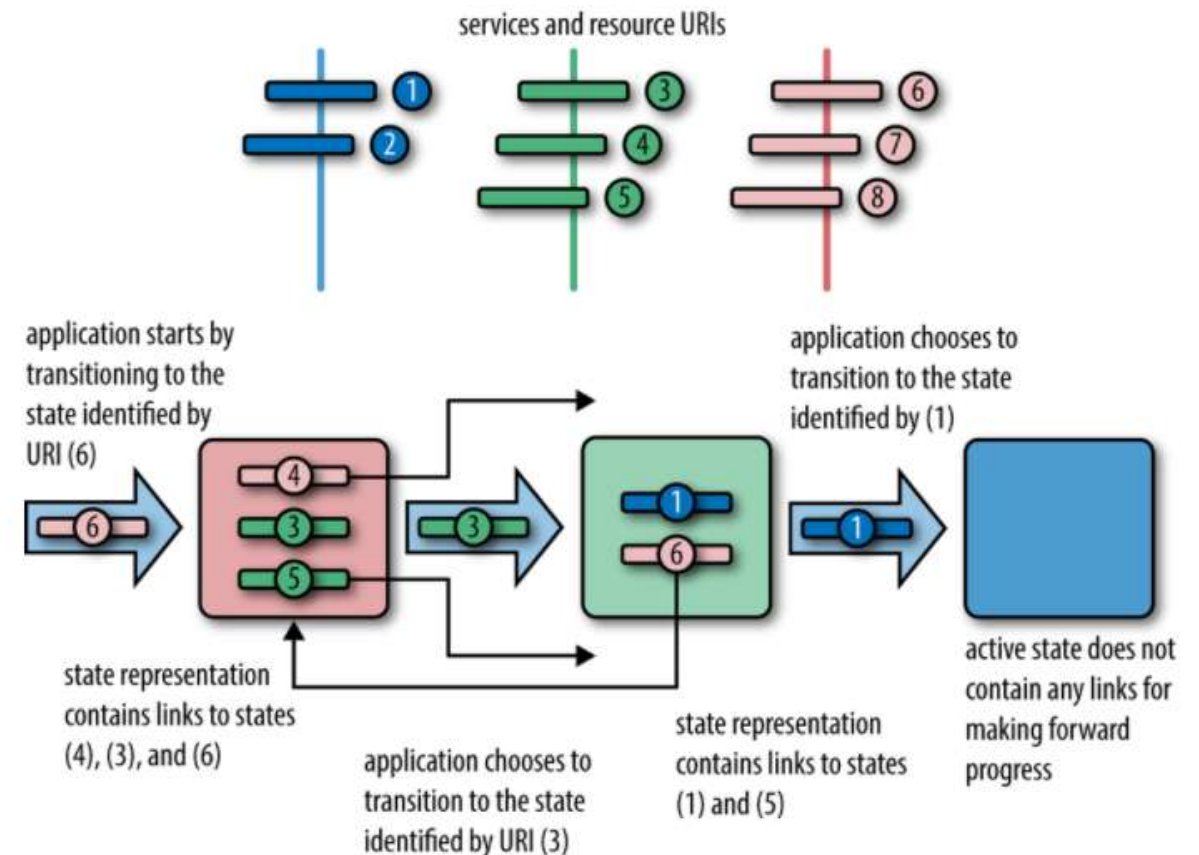
- **Finite-state Machines**

- have a finite number of states
- A state is reached
 - the server transfers a representation of the resource
 - hypermedia is resource representations



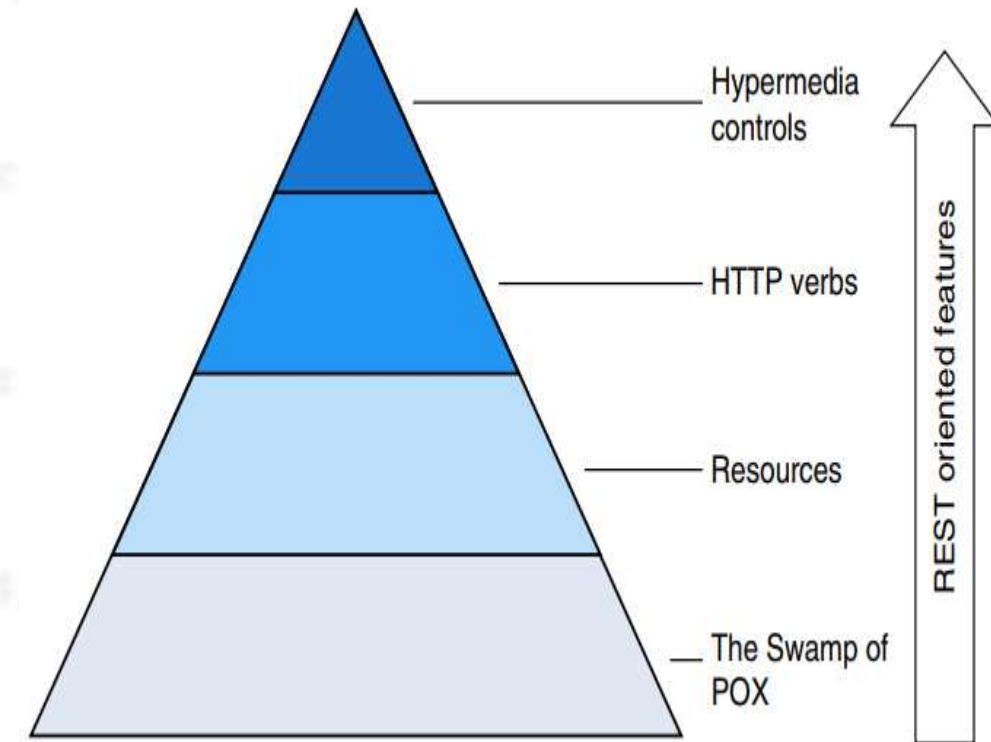
Hypermedia as the Engine of Application State

- hypermedia as the engine of application state (HATEOAS)
 - Include all the information to drive the client
 - RESTful applications progress
 - The client starts from an entry URI or a bookmark
 - The response to a GET request includes a hypermedia representation.
 - The representation contains links to the next states of the FSM
 - Triggers a transition to the next state
 - The client can also go back.



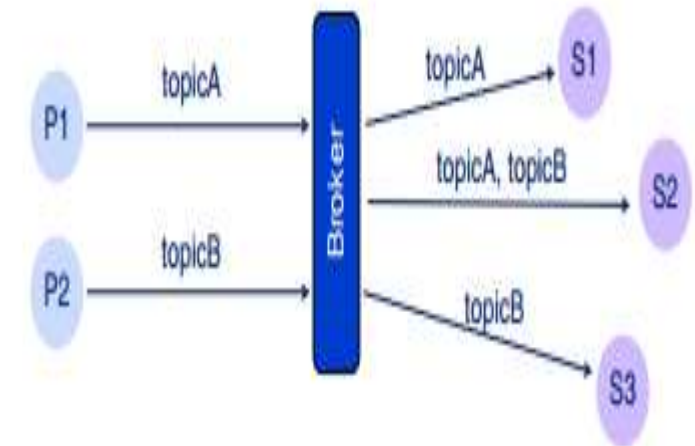
Richardson Maturity Model

- A classification system for web-based applications
 - the higher the level less coupling exists between clients and servers
 - **Level 0: the Swamp of POX**
 - **Level 1: Resources**
 - **Level 2: HTTP Verbs**
 - **Level 3: Hypermedia**



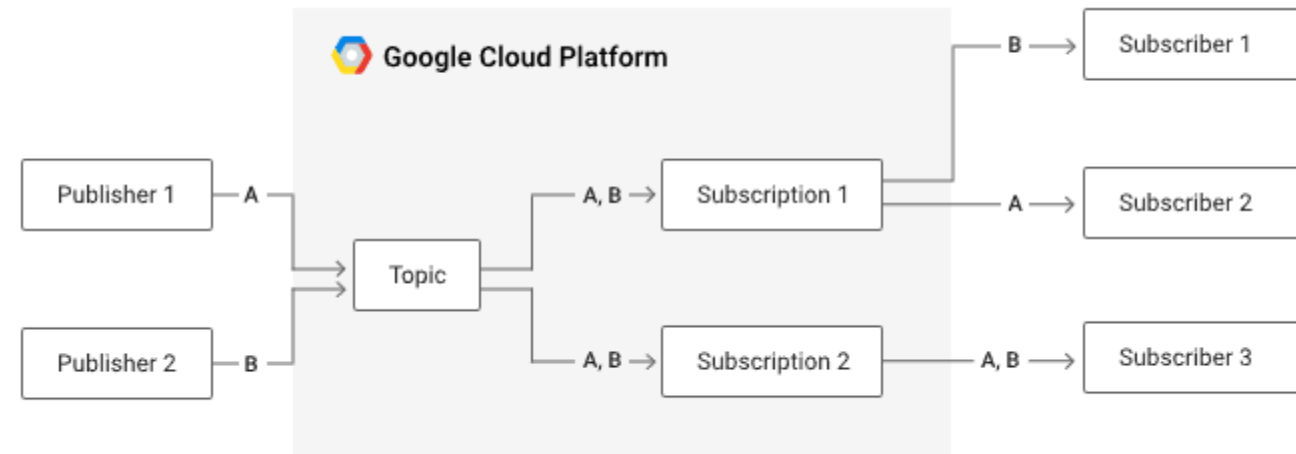
Messaging Queues and Publish/Subscribe Communications

- Implement an asynchronous communication model
 - Message queues
 - a message to a queue on a server
 - kept in memory until a consumer receives
 - Pub/sub
 - Pub send messages to a “topic” on the server
 - Sub can subscribe to a topic to receive a copy of all messages
 - Possible thanks to intermediary nodes (Brokers)
 - *publishing*: publishers send messages to the broker
 - *subscriptions*: subscribers register to receive messages, possibly filtered according to some policy (content or topic).



Advantages of the Pub/Sub Model

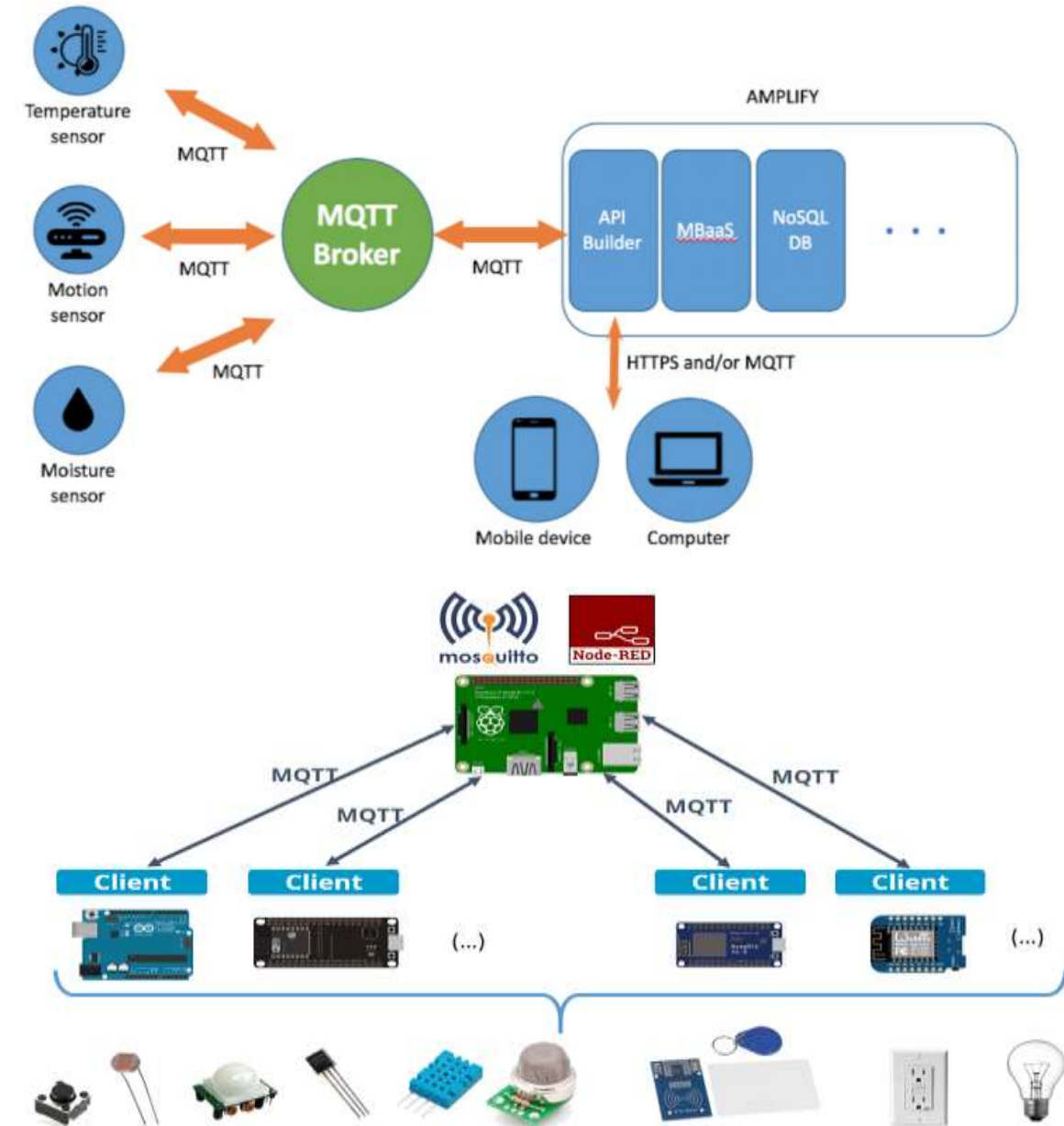
- *Loose coupling*
 - *Be Independent between pub/sub*
- *Scalability*
 - *Support high volumn data*
- *Lightweight implementation*
 - Minimal footprints for sending and receiving messages



Disadvantages of the Pub/Sub Model

Message Queue Telemetry Transport

- MQTT
 - a lightweight protocol
 - open-source
 - TCP-based pub/sub protocol
- Messages are published to a shared topic space inside the MQTT
- Supports hierarchical topics
 - a topic/sub
 - topic/sub-sub-topic path



Advanced message queuing protocols (AMQP)

- AMQP

- Point-to-point
- publish-subscribe
- Be ambility to control flow
- Be compatible to variable programming language

